

Practical 4

Name: Utkarsha Khadse

PRN: 202501040050

4.1.1 Pandas – Series Creation and Manipulation

Problem Statement:

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Code:

```
import pandas as pd
```

```
# Take inputs from the user to create a list of numbers
numbers = list(map(int, input().split()))
```

```
# Create a Pandas series from the list of numbers
series = pd.Series(numbers)
```

```
# Grouping by even and odd numbers and calculating the mean
grouped = series.groupby(series % 2 == 0).mean()
```

```
# Display the mean of even and odd numbers with labels
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
print("Mean of even and odd numbers:")
print(grouped)
```

Output:

4.1.1. Pandas - series creation and manipulation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases

Test Results:

- Average time: 0.015 s, Maximum time: 0.034 s
- 3 out of 3 shown test case(s) passed
- 3 out of 3 hidden test case(s) passed

Test case 1: Expected output: 1 2 3 4 5 6 7 8 9 10, Actual output: 1 2 3 4 5 6 7 8 9 10, Mean of even and odd numbers: 5.0, Odd: 5.0, Even: 6.0, dtype: float64

Test case 2: Expected output: 4 4 4 6 6 2 2 2 10 12 14 16, Actual output: 4 4 4 6 6 2 2 2 10 12 14 16, Mean of even and odd numbers: 6.833333, Odd: 6.833333, Even: 6.0, dtype: float64

Test case 3: Expected output: 4 4 4 6 6 2 2 2 10 12 14 16, Actual output: 4 4 4 6 6 2 2 2 10 12 14 16, Mean of even and odd numbers: 6.833333, Odd: 6.833333, Even: 6.0, dtype: float64

4.1.2 Dictionary to Dataframe

Problem Statement:

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the **Age** column.
- Display the DataFrame after deleting the column.

Code:

```
import pandas as pd
```

```
# Provided dictionary of lists
```

```
data = {  
    'Name': ['Alice', 'Bob', 'Charlie'],  
    'Age': [25, 30, 35],  
}
```

```
# Convert the dictionary to a DataFrame
```

```

df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Adding a new
Name = input("New name: ")
Age = int(input("New age: "))
new = {'Name':Name, 'Age':Age}
df.loc[len(df)] = new
# Display the DataFrame after adding a new row
print("After adding a row:\n",df)

# Modifying a row
index = int(input("Index of row to modify: "))
age = int(input("New age: "))
df.at[index, 'Age']=age
# Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)

# Deleting a row
delt = int(input("Index of row to delete: "))
df = df.drop(delt).reset_index(drop=True)
# Display the DataFrame after deleting a row
print("After deleting a row:")
print(df)

# Adding a new column
gen = input("Enter genders separated by space: ").split()
df['Gender']=gen
# Display the DataFrame after adding a new column
print("After adding a new column:")
print(df)

# Modifying a column
df['Name']=df['Name'].str.upper()
# Display the DataFrame after modifying a column
print("After modifying a column:")
print(df)

# Deleting a column

```

```
df = df.drop(columns = ['Age'])\
# Display the DataFrame after deleting a column
print("After deleting a column:")
print(df)
```

Output:

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column Gender with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Test Case Results:

- 1 out of 1 shown test case(s) passed
- 1 out of 1 hidden test case(s) passed

Debugger:

After deleting a row:

```

Name Age
0 Alice 27
1 Bob 30
2 Charlie 35

```

After adding a new column:

```

Name Age Gender
0 Alice 27 F
1 Bob 30 M
2 Charlie 35 M

```

After modifying a column:

```

Name Age Gender
0 ALICE 27 F
1 BOB 30 M
2 CHARLIE 35 M

```

After deleting a column:

```

Name Gender
0 ALICE F
1 BOB M
2 CHARLIE M

```

4.1.3 Student Information

Problem Statement:

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Code:

```
import pandas as pd

# Read the text file into a DataFrame
file = input()
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])
# write your code here..
print("First five rows:")
print(data.head())
average_age = round(data["Age"].mean(), 2)
print(f"Average age: {average_age}")
filtered_students = data[data["Grade"] <= "B"]
print("Students with a grade up to B")
print(filtered_students)
```

Output:

The screenshot displays a web-based IDE interface for a course titled "4.1.3. Student Information". The main area shows the problem statement and the code to be executed. The code reads a file, displays the first five rows, calculates the average age, and filters students by grade. The output section shows the results of the execution, including the first five rows of data, the average age (23.29), and the filtered list of students with grade B or below.

Expected output:

```
studentdata.txt
First five rows:
-- Name -- Age -- Grade --
0: John -- 25 -- A --
1: Alin -- 22 -- B --
2: Emma -- 24 -- A --
3: Mike -- 23 -- C --
4: Sony -- 21 -- B --
Average age: 23.29
Students with a grade up to B:
-- Name -- Age -- Grade --
0: John -- 25 -- A --
1: Alin -- 22 -- B --
2: Emma -- 24 -- A --
3: Sony -- 21 -- B --
4: Sony -- 21 -- B --
5: Ania -- 26 -- A --
```

Actual output:

```
studentdata.txt
First five rows:
-- Name -- Age -- Grade --
0: John -- 25 -- A --
1: Alin -- 22 -- B --
2: Emma -- 24 -- A --
3: Mike -- 23 -- C --
4: Sony -- 21 -- B --
Average age: 23.29
Students with a grade up to B:
-- Name -- Age -- Grade --
0: John -- 25 -- A --
1: Alin -- 22 -- B --
2: Emma -- 24 -- A --
3: Sony -- 21 -- B --
4: Sony -- 21 -- B --
5: Ania -- 26 -- A --
```

4.2.1 Month with the Highest Total Sales

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Code:

```
import pandas as pd
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
df['Date'] = pd.to_datetime(df['Date'])
df['Total_Sales_Amount'] = df['Quantity'] * df['Price']
Monthly_Sales = df.groupby(df['Date'].dt.strftime('%Y-%m'))['Total_Sales_Amount'].sum()

# Find the month with the highest total sales
best_month = Monthly_Sales.idxmax()
highest_sales = Monthly_Sales.max()
print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```

Output

4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	5	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

```

1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 df['Date'] = pd.to_datetime(df['Date'])
9 df['Month'] = df['Date'].dt.to_period('M').astype(str)
10 df['Total Sales'] = df['Quantity'] * df['Price']
11 monthly_sales = df.groupby('Month')['Total Sales'].sum()
  
```

Test Results: 1 out of 1 shown test case(s) passed, 2 out of 2 hidden test case(s) passed.

Test case 1 (60 ms): Expected output: sales_data.csv, Actual output: sales_data.csv. Best month: 2025-01, Total sales: \$1210.00.

4.2.2 Best Selling Product

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Code:

```

import pandas as pd

# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
product_sales = df.groupby("Product")["Quantity"].sum()
# Find the product with the highest total quantity sold
best_product = product_sales.idxmax()
highest_quantity = product_sales.max()
# Display the result
print(f"Best selling product: {best_product}")
print(f"Total quantity sold: {highest_quantity}")
  
```

Output:

4.2.3 City that Sold the Most Products

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Code:

```
import pandas as pd

# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
# write the code..
city_sales = df.groupby("City")["Quantity"].sum()
best_city = city_sales.idxmax()

# Display the result
print(f"City sold the most products: {best_city}")
```

Output:

4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Code:

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 city_sales = df.groupby('City')['Quantity'].sum()
9 best_city = city_sales.idxmax()
10
11 # Display the result
```

Test Results:

- 1 out of 1 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test Case 1:

Expected output	Actual output
sales_data.csv	sales_data.csv
City sold the most products: Los Angeles	City sold the most products: Los Angeles

4.2.4 Most Frequently Sold Product Pairs

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Code:

```
import pandas as pd
```

```
from itertools import combinations
```

```
from collections import Counter
```

```
# Prompt user to input the file name
```

```
file_name = input()
```

```
# Read data from the specified CSV file
```

```
df = pd.read_csv(file_name)
```

```
# write the code
```

```
grouped = df.groupby("Date")["Product"].apply(list)
```

```

pairs_counter = Counter()
for products in grouped:
    pairs = combinations(sorted(products), 2)
    pairs_counter.update(pairs)

max_frequency = max(pairs_counter.values())
most_common_pairs = [(pair, freq) for pair, freq in pairs_counter.items() if freq ==
max_frequency]

# Output the most frequent product pairs
for (product1, product2), frequency in most_common_pairs:
    print(f"{product1} and {product2}: {frequency} times")

```

Output:

The screenshot displays the CodeTANTRA web application. On the left, a problem titled "4.2.4. Most Frequently Sold Product Pairs" is shown. It asks for a Python program that reads a CSV file, finds pairs of products sold together on the same date, and outputs the most frequent pairs. Sample data and an explanation of the transactions are provided.

On the right, the solution code is shown in a code editor. The code imports pandas, itertools, and collections. It prompts the user for a file name, reads the CSV file, and then uses combinations and Counter to find the most frequent product pairs.

Below the code, test results are displayed. Two test cases passed: "Test case 1" (76 ms) and "Test case 2" (76 ms). The expected output and actual output are shown for each test case.

The bottom of the screen shows a Windows taskbar with various application icons and system information.

4.2.5 Titanic Dataset Analysis and Data Cleaning

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
print(data.head())
```

```
# 1. Display the first 5 rows of the dataset
print(data.tail())
```

```
# 2. Display the last 5 rows of the dataset
print(data.shape)
```

```
# 3. Get the shape of the dataset
print(data.info())
```

```
# 4. Get a summary of the dataset (info)
print(data.describe())
```

```
# 5. Get basic statistics of the dataset
print(data.isnull().sum())
```

```
# 6. Check for missing value
median_age = data['Age'].median()
data['Age'].fillna(median_age, inplace = True)

# 7. Fill missing values in the 'Age' column with the median age
mode_embarked = data['Embarked'].mode() [0]
data['Embarked'].fillna(mode_embarked, inplace = True)

# 8. Fill missing values in the 'Embarked' column with the mode
data.drop('Cabin', axis=1, inplace = True)

# 9. Drop the 'Cabin' column due to many missing values
data['FamilySize'] = data['SibSp'] + data['Parch']
# 10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'
```

Output:

64° WhatsApp

Course

Ask Gemini

mitaoe.codeatntra.com/secure/course.jsp?exid=698c5486aba96214c8b7d5e#contents/698c5486aba96214c8b7d5e/67e2b30925661df3as574f

20251040050@mitaoe.ac.in Support Logout

4.2.5. Titanic Dataset Analysis and Data Cleaning

0.223 s

223.0 ms

1 out of 1 shown test case(s) passed

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

- Display the first 5 rows of the dataset.
- Display the last 5 rows of the dataset.
- Get the shape of the dataset (number of rows and columns).
- Get a summary of the dataset (using `info()`).
- Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
- Check for missing values and display the count of missing values for each column.
- Fill missing values in the 'Age' column with the median age.
- Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
- Drop the 'Cabin' column due to many missing values.
- Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Braund	male	22	1	0	151601	5.42	NaN	S
2	1	1	Mr. Allen	male	35	1	0	151706	7.25	NaN	C
3	0	3	Mr. Brown	male	29	0	0	151709	5.39	NaN	S
4	1	1	Mr. Davis	male	35	1	0	151744	7.12	NaN	C
5	0	3	Mr. Miller	male	22	1	0	151786	5.51	NaN	S

Sample Data:

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Braund	male	22	1	0	151601	5.42	NaN	S
2	1	1	Mr. Allen	male	35	1	0	151706	7.25	NaN	C
3	0	3	Mr. Brown	male	29	0	0	151709	5.39	NaN	S
4	1	1	Mr. Davis	male	35	1	0	151744	7.12	NaN	C
5	0	3	Mr. Miller	male	22	1	0	151786	5.51	NaN	S

64° WhatsApp

Course

Ask Gemini

mitaoe.codeatntra.com/secure/course.jsp?exid=698c5486aba96214c8b7d5e#contents/698c5486aba96214c8b7d5e/67e2b30925661df3as574f

20251040050@mitaoe.ac.in Support Logout

4.2.5. Titanic Dataset Analysis and Data Cleaning

0.223 s

223.0 ms

1 out of 1 shown test case(s) passed

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

- Display the first 5 rows of the dataset.
- Display the last 5 rows of the dataset.
- Get the shape of the dataset (number of rows and columns).
- Get a summary of the dataset (using `info()`).
- Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
- Check for missing values and display the count of missing values for each column.
- Fill missing values in the 'Age' column with the median age.
- Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
- Drop the 'Cabin' column due to many missing values.
- Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Braund	male	22	1	0	151601	5.42	NaN	S
2	1	1	Mr. Allen	male	35	1	0	151706	7.25	NaN	C
3	0	3	Mr. Brown	male	29	0	0	151709	5.39	NaN	S
4	1	1	Mr. Davis	male	35	1	0	151744	7.12	NaN	C
5	0	3	Mr. Miller	male	22	1	0	151786	5.51	NaN	S

Sample Data:

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Braund	male	22	1	0	151601	5.42	NaN	S
2	1	1	Mr. Allen	male	35	1	0	151706	7.25	NaN	C
3	0	3	Mr. Brown	male	29	0	0	151709	5.39	NaN	S
4	1	1	Mr. Davis	male	35	1	0	151744	7.12	NaN	C
5	0	3	Mr. Miller	male	22	1	0	151786	5.51	NaN	S

64° WhatsApp

Course

Ask Gemini

mitaoe.codeatntra.com/secure/course.jsp?exid=698c5486aba96214c8b7d5e#contents/698c5486aba96214c8b7d5e/67e2b30925661df3as574f

20251040050@mitaoe.ac.in Support Logout

4.2.5. Titanic Dataset Analysis and Data Cleaning

0.223 s

223.0 ms

1 out of 1 shown test case(s) passed

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

- Display the first 5 rows of the dataset.
- Display the last 5 rows of the dataset.
- Get the shape of the dataset (number of rows and columns).
- Get a summary of the dataset (using `info()`).
- Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
- Check for missing values and display the count of missing values for each column.
- Fill missing values in the 'Age' column with the median age.
- Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
- Drop the 'Cabin' column due to many missing values.
- Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Braund	male	22	1	0	151601	5.42	NaN	S
2	1	1	Mr. Allen	male	35	1	0	151706	7.25	NaN	C
3	0	3	Mr. Brown	male	29	0	0	151709	5.39	NaN	S
4	1	1	Mr. Davis	male	35	1	0	151744	7.12	NaN	C
5	0	3	Mr. Miller	male	22	1	0	151786	5.51	NaN	S

Sample Data:

PassengerId	Survived	Pclass	Name
-------------	----------	--------	------

4.2.6 Titanic Dataset Analysis and Data Cleaning-2

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
data['Alone'] = np.where(data['FamilySize'] == 0, 1, 0)
```

```
# 2. Convert 'Sex' to numeric (male: 0, female: 1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
```

```
# 3. One-hot encode the 'Embarked' column
data = pd.get_dummies(data, columns = ['Embarked'], drop_first = True)
```

```
# 4. Get the mean age of passengers
mean_age = data['Age'].mean()
print(mean_age)
```

```
# 5. Get the median fare of passengers
median_fare = data['Fare'].median()
```



```
print(median_fare)
```

6. Get the number of passengers by class

```
passengers_by_class = data['Pclass'].value_counts()
print(passengers_by_class)
```

7. Get the number of passengers by gender

```
passengers_by_gender = data['Sex'].value_counts().sort_index()
print(passengers_by_gender)
```

8. Get the number of passengers by survival status

```
passengers_by_survival = data['Survived'].value_counts().sort_index()
print(passengers_by_survival)
```

9. Calculate the survival rate

```
survival_rate = data['Survived'].mean()
print(survival_rate)
```

10. Calculate the survival rate by gender

```
survival_rate_by_gender = data.groupby('Sex')['Survived'].mean()
print(survival_rate_by_gender)
```

Output:

The screenshot displays a web-based coding environment with a task description, a sample data table, and a test case output.

Task Description: 4.2.6. Titanic Dataset Analysis and Data Cleaning - 2. You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	ParCh	Ticket	Fare	Cabin	Embarked

Sample Data:

Test Case 1 Output:

Expected output	Actual output
29.69911764705882	29.69911764705882
14.4542	14.4542
3.491	3.491
1.216	1.216
2.184	2.184
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64
0.577	0.577
1.314	1.314
Name: Sex, dtype: int64	Name: Sex, dtype: int64
0.549	0.549
1.342	1.342
Name: Survived, dtype: int64	Name: Survived, dtype: int64
0.3838383838383838	0.3838383838383838
Sex	Sex
0.188988	0.188988
1.742038	1.742038
Name: Survived, dtype: float64	Name: Survived, dtype: float64

4.2.7 Titanic Dataset Analysis and Data Cleaning-3

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

Code:

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Calculate the survival rate by class
print(data.groupby('Pclass')['Survived'].mean())

# 2. Calculate the survival rate by embarked location
print(data.groupby('Embarked_S')['Survived'].mean())

# 3. Calculate the survival rate by family size
print(data.groupby('FamilySize')['Survived'].mean())

# 4. Calculate the survival rate by being alone
print(data.groupby('IsAlone')['Survived'].mean())

# 5. Get the average fare by class
print(data.groupby('Pclass')['Fare'].mean())

# 6. Get the average age by class
```

```
print(data.groupby('Pclass')['Age'].mean())
```

7. Get the average age by survival status

```
print(data.groupby('Survived')['Age'].mean())
```

8. Get the average fare by survival status

```
print(data.groupby('Survived')['Fare'].mean())
```

9. Get the number of survivors by class

```
print(data[data['Survived'] == 1]['Pclass'].value_counts())
```

10. Get the number of non-survivors by class

```
print(data[data['Survived'] == 0]['Pclass'].value_counts())
```

Output:

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (isAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	ParCh	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Brown	Male	34	1	0	12345	53.1	B5	S
2	1	1	Mr. Green	Male	42	0	0	67890	51.0	C123	C
3	0	3	Mr. Blue	Male	18	1	0	11111	16.99	D	Q
4	0	3	Mr. Black	Male	26	0	0	22222	21.01	E	S
5	0	3	Mr. Grey	Male	30	1	0	33333	23.68	F	S
6	0	3	Mr. White	Male	38	0	0	44444	30.09	G	S
7	0	3	Mr. Red	Male	45	1	0	55555	36.33	H	S
8	0	3	Mr. Yellow	Male	52	0	0	66666	43.47	I	S
9	0	3	Mr. Purple	Male	59	1	0	77777	50.49	J	S
10	0	3	Mr. Brown	Male	66	0	0	88888	57.51	K	S

Sample Data:

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	ParCh	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Brown	Male	34	1	0	12345	53.1	B5	S
2	1	1	Mr. Green	Male	42	0	0	67890	51.0	C123	C
3	0	3	Mr. Blue	Male	18	1	0	11111	16.99	D	Q
4	0	3	Mr. Black	Male	26	0	0	22222	21.01	E	S
5	0	3	Mr. Grey	Male	30	1	0	33333	23.68	F	S
6	0	3	Mr. White	Male	38	0	0	44444	30.09	G	S
7	0	3	Mr. Red	Male	45	1	0	55555	36.33	H	S
8	0	3	Mr. Yellow	Male	52	0	0	66666	43.47	I	S
9	0	3	Mr. Purple	Male	59	1	0	77777	50.49	J	S
10	0	3	Mr. Brown	Male	66	0	0	88888	57.51	K	S

Test case 1: 1 out of 1 shown test case(s) passed

Expected output	Actual output
Pclass	Pclass
1... 0.529638	1... 0.529638
2... 0.473264	2... 0.473264
3... 0.242263	3... 0.242263
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Embarked_S	Embarked_S
0... 0.586873	0... 0.586873
1... 0.338957	1... 0.338957
Name: Survived, dtype: float64	Name: Survived, dtype: float64
FamilySize	FamilySize
0... 0.385538	0... 0.385538
1... 0.552795	1... 0.552795
2... 0.578431	2... 0.578431
3... 0.724138	3... 0.724138
4... 0.280000	4... 0.280000
5... 0.196364	5... 0.196364
6... 0.333333	6... 0.333333
7... 0.400000	7... 0.400000
8... 0.400000	8... 0.400000
9... 0.400000	9... 0.400000
10... 0.400000	10... 0.400000

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (isAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	ParCh	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Brown	Male	34	1	0	12345	53.1	B5	S
2	1	1	Mr. Green	Male	42	0	0	67890	51.0	C123	C
3	0	3	Mr. Blue	Male	18	1	0	11111	16.99	D	Q
4	0	3	Mr. Black	Male	26	0	0	22222	21.01	E	S
5	0	3	Mr. Grey	Male	30	1	0	33333	23.68	F	S
6	0	3	Mr. White	Male	38	0	0	44444	30.09	G	S
7	0	3	Mr. Red	Male	45	1	0	55555	36.33	H	S
8	0	3	Mr. Yellow	Male	52	0	0	66666	43.47	I	S
9	0	3	Mr. Purple	Male	59	1	0	77777	50.49	J	S
10	0	3	Mr. Brown	Male	66	0	0	88888	57.51	K	S

Sample Data:

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	ParCh	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Brown	Male	34	1	0	12345	53.1	B5	S
2	1	1	Mr. Green	Male	42	0	0	67890	51.0	C123	C
3	0	3	Mr. Blue	Male	18	1	0	11111	16.99	D	Q
4	0	3	Mr. Black	Male	26	0	0	22222	21.01	E	S
5	0	3	Mr. Grey	Male	30	1	0	33333	23.68	F	S
6	0	3	Mr. White	Male	38	0	0	44444	30.09	G	S
7	0	3	Mr. Red	Male	45	1	0	55555	36.33	H	S
8	0	3	Mr. Yellow	Male	52	0	0	66666	43.47	I	S
9	0	3	Mr. Purple	Male	59	1	0	77777	50.49	J	S
10	0	3	Mr. Brown	Male	66	0	0	88888	57.51	K	S

Test case 1: 1 out of 1 shown test case(s) passed

Expected output	Actual output
2... 20.662183	2... 20.662183
3... 13.675558	3... 13.675558
Name: Fare, dtype: float64	Name: Fare, dtype: float64
Pclass	Pclass
1... 38.233441	1... 38.233441
2... 29.877630	2... 29.877630
3... 25.148620	3... 25.148620
Name: Age, dtype: float64	Name: Age, dtype: float64
Survived	Survived
0... 30.626179	0... 30.626179
1... 28.343690	1... 28.343690
Name: Age, dtype: float64	Name: Age, dtype: float64
Survived	Survived
0... 22.117887	0... 22.117887
1... 48.395408	1... 48.395408
Name: Fare, dtype: float64	Name: Fare, dtype: float64
1... 136	1... 136
3... 119	3... 119
2... 87	2... 87
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64
3... 372	3... 372
2... 97	2... 97

4.2.8 Titanic Dataset Analysis and Data Cleaning-4

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# 1. Get the number of survivors by gender
```

```
survivors_by_gender = data[data['Survived'] == 1]['Sex'].value_counts()
```

```
print(survivors_by_gender)
```

```
# 2. Get the number of non-survivors by gender
```

```
non_survivors_by_gender = data[data['Survived'] == 0]['Sex'].value_counts()
```

```
print(non_survivors_by_gender)
```

```
# 3. Get the number of survivors by embarked location
```

```
survivors_by_embarked_s = data[data['Survived'] ==
```

```
1]['Embarked_S'].value_counts()
```

```
print(survivors_by_embarked_s)
```

```
# 4. Get the number of non-survivors by embarked location
```

```
non_survivors_by_embarked_s = data[data['Survived'] ==
```

```
0]['Embarked_S'].value_counts()
```

```
print(non_survivors_by_embarked_s)
```

5. Calculate the percentage of children (Age < 18) who survived

```
children = data[data['Age'] < 18]
children_survival_rate = children['Survived'].mean()
print(children_survival_rate)
```

6. Calculate the percentage of adults (Age >= 18) who survived

```
adults = data[data['Age'] >= 18]
adults_survival_rate = adults['Survived'].mean()
print(adults_survival_rate)
```

7. Get the median age of survivors

```
median_age_survivors = data[data['Survived'] == 1]['Age'].median()
print(median_age_survivors)
```

8. Get the median age of non-survivors

```
median_age_non_survivors = data[data['Survived'] == 0]['Age'].median()
print(median_age_non_survivors)
```

9. Get the median fare of survivors

```
median_fare_survivors = data[data['Survived'] == 1]['Fare'].median()
print(median_fare_survivors)
```

10. Get the median fare of non-survivors

```
median_fare_non_survivors = data[data['Survived'] == 0]['Fare'].median()
print(median_fare_non_survivors)
```

Output:

The screenshot displays a web-based coding environment with a task description on the left and a code execution interface on the right.

Task Description:

4.2.8. Titanic Dataset Analysis and Data Cleaning -4

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	ParCh	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Brown, Mrs. Brown	male	22.0	1	0	123456	53.1	C123	S
2	1	1	Mr. Gold	male	42.0	0	0	789012	51.68	A987	C
3	0	3	Mr. Miller	male	35.0	1	0	345678	16.99	F567	Q
4	1	2	Mr. Davis	male	28.0	0	0	901234	26.33	D456	S
5	0	3	Mr. Wilson	male	18.0	0	0	567890	5.44	E123	Q
6	1	1	Mr. White	male	58.0	0	0	234567	51.68	A987	C
7	0	3	Mr. Green	male	25.0	1	0	890123	16.99	F567	Q
8	1	2	Mr. Black	male	32.0	0	0	456789	26.33	D456	S
9	0	3	Mr. Taylor	male	15.0	0	0	678901	5.44	E123	Q
10	1	1	Mr. King	male	60.0	0	0	123456	51.68	A987	C

Sample Data:

Code Execution Interface:

Test case 1 (113 ms) [Debug]

Expected output:

```
female: 233
male: 109
Name: Sex, dtype: int64
male: 468
female: 81
Name: Sex, dtype: int64
1: 217
0: 125
Name: Embarked_S, dtype: int64
1: 427
0: 122
Name: Embarked_S, dtype: int64
0.539823088495575
0.3818316139767855
28.0
28.0
26.0
10.5
```

Actual output:

```
female: 233
male: 109
Name: Sex, dtype: int64
male: 468
female: 81
Name: Sex, dtype: int64
1: 217
0: 125
Name: Embarked_S, dtype: int64
1: 427
0: 122
Name: Embarked_S, dtype: int64
0.539823088495575
0.3818316139767855
28.0
28.0
26.0
10.5
```

1 out of 1 shown test case(s) passed

